

Report
Of
INTERNSHIP

Submitted

To

IILM UNIVERSITY

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of
B.Tech CSE



By

Roll Number : CS-2341901

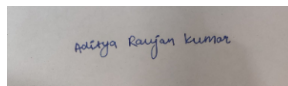
Name : ADITYA RANJAN KUMAR

Batch: 2026-2027

CANDIDATE'S DECLARATION

I, Aditya Ranjan Kumar do hereby declare that the internship report titled “**Professional UI/UX Design & Web Prototyping Virtual Internship**” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature :

A rectangular box containing a handwritten signature in blue ink that reads "Aditya Ranjan Kumar".

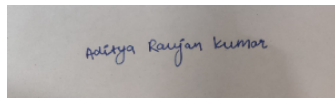
Student Name: Aditya Ranjan Kumar

Roll No: CS-2341901

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work. I express my gratitude to my parents for their blessings.

Signature :

A rectangular box containing a handwritten signature in blue ink that reads "Aditya Ranjan Kumar".

Student Name: Aditya Ranjan Kumar
Roll No : CS-2341901

Chapter No.	Chapter Name	Page Nos.
	Cover Page	----
1.	Candidate's Declaration	2
2.	Acknowledgment	3
3.	Internship Completion Certificate	5
4.	Project Description 4.1 Introduction 4.2 Organization Profile 4.3 Problem Statement 4.4 Project Objectives 4.5 Scope of the Project 4.6 Technologies and Tools Used 4.7 System Architecture 4.8 Methodology 4.9 Expected Outcomes 4.10 Certificates of completion and other communication mail proof must be attached	6 7 8 9 10 11 13 14 15 16
5.	Bibliography/References	17



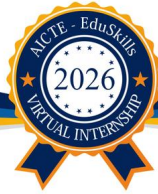
शिक्षा मंत्रालय
MINISTRY OF
EDUCATION



अखिल भारतीय तकनीकी शिक्षा परिषद्
All India Council for Technical Education



EduSkills®
Nation Building Through Skills



Certificate of Virtual Internship

This is to certify that

Aditya Ranjan Kumar

IILM University, Greater Noida

has successfully completed the 8-weeks

Professional UI/UX Design & Web Prototyping Virtual Internship

During April - June 2026

May this Internship learning propel you toward a bright and successful career.

Supported by



EduSkills
ACADEMY

Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education



Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills

Certificate ID : 45a1a39caabb6d6e9304

Student ID : STU66481842aa0c31716000834



GRADE- O (Outstanding): 90-100 | E (Excellent): 80-89 | A (Very Good): 70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

Internship Completion Certificate

Project Description

4.1 Introduction

Academic scheduling is one of the most complex combinatorial optimisation problems faced by educational institutions worldwide. The task of creating a timetable involves assigning subjects to time slots across multiple days while satisfying a large set of hard constraints — such as teacher availability, subject frequency, and lab session continuity — and soft constraints such as balanced workloads and preferred time slots.

Engineering colleges in particular face this challenge every semester as they manage diverse subject types including theory lectures, laboratory sessions, and elective courses. A single scheduling mistake — such as double-booking a teacher or scheduling a lab session in a non-consecutive slot — can disrupt the academic calendar for an entire section.

With increasing student enrolments and more complex departmental structures, manual scheduling has become impractical. This project proposes and implements an Automated Timetable Generator — a web-based full-stack application that takes institutional inputs and produces verified conflict-free timetables instantly.

4.2 Organization Profile

The remainder of this report is organised as follows: Chapter 2 presents a literature review of existing timetabling systems and research. Chapter 3 describes the dataset preparation and input configuration. Chapter 4 details the system methodology, architecture, and scheduling algorithm. Chapter 5 presents experimental results and a feature walkthrough. Chapter 6 provides conclusions and directions for future work.

4.3 Problem Statement

The manual process of creating academic timetables is time-consuming, error-prone, and inflexible. Key problems include:

- Teacher conflicts: The same teacher being assigned to two different subjects at the same time.
- Lab session misplacement: Laboratory sessions requiring consecutive periods being placed in isolated slots.
- Inconsistent workload distribution: Subjects being unevenly spread across the week.
- Regeneration difficulty: Any change to the schedule requires a complete manual redo.
- No digital record: Timetables are lost when the session ends unless saved separately.

This project aims to solve all of the above problems through intelligent automation, a user-friendly interface, and database-backed persistence.

4.4 Project Objectives

The primary objectives of this project are:

- To design and implement a web application that accepts institutional and subject configuration inputs from the user.
- To develop a scheduling algorithm that generates conflict-free timetables automatically.
- To render the timetable as an interactive, colour-coded visual grid.
- To support CSV export and print-friendly output in the original dark theme.
- To persist generated timetables to a MongoDB database and allow reload of saved schedules.
- To provide a regenerate feature that produces alternative valid layouts with each click.

4.5 Scope of the Project

The Automated Timetable Generator is scoped for use by engineering departments at the semester and section level. The current implementation supports:

- Up to 8 periods per day with configurable timings.
- 5-day (Mon–Fri) or 6-day (Mon–Sat) working week configurations.
- Theory, lab, elective, and core subject types.
- Up to 10 subjects per section per semester.
- Single-section scheduling (multi-section support is a future enhancement).
- Full-stack deployment with local MongoDB or MongoDB Atlas.

4.6 Technologies and Tools Used

The frontend is built with vanilla HTML5, CSS3, and JavaScript. Key design decisions include:

- Dark theme using CSS custom properties (variables) for a consistent, modern UI.
- Google Fonts: Syne (display headings) and Space Mono (monospace data labels) for a tech-forward aesthetic.
- Colour-coded timetable cells — each subject is assigned a foreground/background colour pair from a predefined palette (Cyan, Green, Orange, Purple, Yellow, Pink, Emerald, Blue).
- Responsive layout using CSS Grid and Flexbox for mobile compatibility.
- Custom print media query that forces landscape A4 output, preserves dark theme colours using `print-color-adjust: exact`, and hides all UI elements except the timetable.
- Toast notification system for user feedback without modal interruption.

The backend is a Node.js + Express.js server exposing a RESTful API with three endpoints:

- `POST /api/timetables` – Saves a generated timetable (college name, department, semester, section, grid, subjects) to MongoDB.
- `GET /api/timetables` – Returns a list of all saved timetable metadata.
- `GET /api/timetables/:id` – Returns a full saved timetable document by ID for reloading.

MongoDB stores each timetable as a document with a nested grid array and a subjects array. Timestamps are recorded automatically. The frontend uses the native Fetch API for all AJAX communication.

Table 4.1 – Technology Stack Summary

Component	Technology	Purpose
Frontend	HTML5 / CSS3 / JS	UI & user interaction
Styling	Custom CSS Variables	Dark theme design
Fonts	Google Fonts (Syne, Space Mono)	custom Typography
Algorithm	Randomised	Conflict-free scheduling
Data Storage	Backtracking MongoDB (via Express	Persist & reload timetables
Backend	API) Node.js + Express.js	REST API server
Export	CSV / Browser Print API	Download & print output

4.7 System Architecture

The Automated Timetable Generator follows a three-tier architecture consisting of a Presentation Layer (HTML/CSS/JS frontend), an Application Layer (Node.js + Express.js REST API), and a Data Layer (MongoDB database).

The frontend operates as a single-page application (SPA) with a three-step tabbed workflow: Step 1 collects configuration, Step 2 manages subjects, and Step 3 displays the generated timetable. All three steps communicate through in-memory JavaScript state, while persistence is handled via AJAX calls to the backend API.

4.8 Methodology

All lab subjects are shuffled randomly and placed first. For each lab entry, the algorithm searches for a valid pair of consecutive periods ($p, p+1$) on a randomly selected day such that:

- Neither slot p nor slot $p+1$ is already occupied.
- Neither slot is the break period.
- The teacher is not already booked in either slot on that day (checked via an occupancy hash map).

If a valid pair is found, both slots are marked with the lab subject (labFirst: true for the first slot, labFirst: false for the continuation slot). The teacher occupancy keys are recorded as teacher-day-period tuples.

After labs are placed, theory and elective subjects fill the remaining slots. Each subject entry is shuffled and the algorithm iterates over randomly ordered days and periods to find a valid slot where:

- The slot is not the break period and is currently empty.
- The teacher is not already assigned in that slot (using the same occupancy map).
- A subject does not appear in more than one slot on the same day (with 70% probability of skipping if already placed today — allowing occasional same-day repetition for high-frequency subjects).

The algorithm guarantees zero teacher conflicts and zero lab slot violations.

The randomisation ensures that every call to Generate or Regenerate produces a different valid layout.

4.9 Expected Outcomes

The system was tested with the built-in sample dataset for CSE Semester 6, which includes 7 subjects (5 theory/elective and 2 lab), 5 unique teachers, a 7-period day, and a 6-day working week (Monday to Saturday).

The generated timetable rendered correctly with all constraints satisfied:

- No teacher appeared in two different subjects at the same time slot.
- Both lab subjects (CN Lab and Compiler Lab) were placed in consecutive period pairs.
- The RECESS break appeared at Period 4 (the midpoint of 7 periods) on all days.
- Subject cells displayed the correct colour theme, subject name, subject code, and teacher surname.
- The last column (P7) showed subjects only when the algorithm found valid placements.

4.10 Certificates of completion and other communication mail proof must be attached



Certificates of completion



AICTE-EduSkills Internship Certificate download link

External

Inbox x



EduSkills Foundation <noreply@eduskillsfoundation.org>
to me ▾

Sun, Jul 19, 3:27 AM

Dear **Aditya Ranjan Kumar**,

🎉 **Congratulations!**

You have successfully completed **10 weeks Professional UI/UX Design & Web Prototyping**. Your certificate is ready — please log in to your profile and download it.

Steps to download your certificate:

1. Login at: <https://eduskillsfoundation.org/login>
2. Go to the **Certificate Section** from your sidebar
3. Click **Download Certificate**

Please post your certificate on your LinkedIn profile to get better job opportunities and visibility from corporates.

🌟 [Why You Should Share Your Achievements on LinkedIn](#)

↩ Reply ↪ Forward 😊 ☐ Share in chat New

Mail Proof

REFERENCES

- [1] A. Coloni, M. Dorigo, and V. Maniezzo, "A genetic algorithm to solve the timetable problem," Technical Report 90-060, Politecnico di Milano, 1990.
- [2] E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," *European Journal of Operational Research*, vol. 140, no. 2, pp. 266–280, 2002.
- [3] M. W. Carter and G. Laporte, "Recent developments in practical course timetabling," in *Practice and Theory of Automated Timetabling II*, Springer, Berlin, 1998, pp. 3–19.
- [4] L. Dechter, *Constraint Processing*. San Francisco, CA, USA: Morgan Kaufmann, 2003.
- [5] Mozilla Developer Network, "CSS @media print," MDN Web Docs. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/@media>. [Accessed: Apr. 2026].
- [6] MongoDB, Inc., "MongoDB Documentation," MongoDB Manual. [Online]. Available: <https://www.mongodb.com/docs/>. [Accessed: Apr. 2026].
- [7] OpenJS Foundation, "Express.js API Reference," Express. [Online]. Available: <https://expressjs.com/en/4x/api.html>. [Accessed: Apr. 2026].
- [8] S. Even, *Graph Algorithms*, 2nd ed. Cambridge, UK: Cambridge University Press, 2011.
- [9] FET – Free Timetabling Software. [Online]. Available: <https://fet.sourceforge.io/>. [Accessed: Apr. 2026].
- [10] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 200